Kubernetes Cluster Benchmarking Guide

Turing Pi 2.5 — 4× RK1 Compute Modules

Companion to: Turing Pi 2.5 K3s HA Cluster Deployment Guide

February 2026

Version 1.0

Storage Performance • Control Plane Performance • CIS Security Compliance

1. Introduction

This guide covers three complementary benchmarking tools for evaluating the performance and security posture of a K3s cluster running on Turing Pi 2.5 hardware. It assumes you have already completed the Turing Pi 2.5 K3s HA Cluster Deployment Guide and have a fully functional cluster with Longhorn storage, MetalLB, and monitoring installed.

The three benchmarks are:

Longhorn kbench (Storage Benchmark): Uses FIO to measure NVMe storage performance through Longhorn's distributed storage layer. Reports IOPS, bandwidth, and latency for both random and sequential workloads. Compares Longhorn performance against local-path-provisioner to quantify the distributed storage overhead.

Control Plane Benchmark: Measures Kubernetes control plane responsiveness by timing pod lifecycle operations (deploy, scale out, scale in, delete) using standard kubectl commands. No additional tools required.

Aqua kube-bench (CIS Security Benchmark): Audits the cluster configuration against Center for Internet Security (CIS) Kubernetes Benchmark guidelines. Identifies security misconfigurations and provides remediation guidance. Includes K3s-specific benchmark profiles.

1.1 Prerequisites

Before running any benchmarks, verify the cluster is healthy:

```
# All 4 nodes should show Ready
kubectl get nodes

# No pods in error state
kubectl get pods -A | grep -v Running | grep -v Completed

# etcd is healthy
kubectl get --raw=/healthz/etcd

# Longhorn volumes are healthy
kubectl -n longhorn-system get volumes.longhorn.io
```

If any of these checks fail, resolve the issues using the troubleshooting procedures in the deployment guide before benchmarking. Results from an unhealthy cluster are not meaningful.

1.2 Hardware Under Test

For reference, the Turing Pi 2.5 cluster being benchmarked consists of:

Compute: 4× Turing RK1 modules (Rockchip RK3588, 8-core ARM64: 4× Cortex-A76 + 4× Cortex-A55, 32GB RAM each)

Storage: 4× 1TB NVMe M.2 2280 SSDs (one per node, ~295GB reserved for Longhorn at 30%)

Network: 1 Gbps Ethernet per node via Turing Pi 2.5 backplane

Software: Ubuntu 24.04, K3s v1.34.x, Longhorn v1.11.0, dual-stack IPv4/IPv6

2. Storage Benchmark: Longhorn kbench

Longhorn's kbench tool (github.com/longhorn/kbench) uses FIO, the industry-standard flexible I/O tester, to benchmark Kubernetes storage classes. It measures IOPS, bandwidth, and latency across random and sequential read/write patterns.

2.1 Why Benchmark Storage?

Longhorn is a distributed storage system that replicates data across nodes for redundancy. This replication introduces overhead compared to writing directly to a local NVMe drive. Understanding this overhead is important for capacity planning: if your workloads are I/O-intensive, you need to know how much performance Longhorn trades for its data protection guarantees.

The kbench tool runs a comparison benchmark that tests Longhorn and local-path-provisioner side by side, giving you a direct measurement of the distributed storage overhead on your specific hardware.

2.2 Run a Single Storage Class Benchmark (Longhorn)

Important — ARM64 compatibility: The official yasker/kbench image is built for x86/amd64 only and will fail with 'exec format error' on the RK1's ARM64 processors. The manifests below use Alpine Linux with FIO installed directly, which supports ARM64 natively.

This tests Longhorn storage in isolation. The job creates a PVC, runs a comprehensive FIO benchmark against it, and reports results.

```
# Create the benchmark job manifest
cat <<'EOF' > kbench-longhorn.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: kbench-pvc-longhorn
spec:
  storageClassName: longhorn
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 30Gi
---
apiVersion: batch/v1
kind: Job
metadata:
  name: kbench
spec:
  template:
    metadata:
      labels:
        kbench: fio
    spec:
```

```
      containers:
      - name: kbench
        image: alpine:latest
        command: ["/bin/sh", "-c"]
        args:
        - |
          apk add --no-cache fio
          echo "=== Longhorn Storage Benchmark ==="
          echo "=== Random Read IOPS (4K) ==="
          fio --name=rand-read --ioengine=libaio --iodepth=64 --rw=randread \
              --bs=4k --direct=1 --size=2G --numjobs=1 --runtime=60 \
              --group_reporting --directory=/volume
          echo ""
          echo "=== Random Write IOPS (4K) ==="
          fio --name=rand-write --ioengine=libaio --iodepth=64 --rw=randwrite \
              --bs=4k --direct=1 --size=2G --numjobs=1 --runtime=60 \
              --group_reporting --directory=/volume
          echo ""
          echo "=== Sequential Read Bandwidth (128K) ==="
          fio --name=seq-read --ioengine=libaio --iodepth=64 --rw=read \
              --bs=128k --direct=1 --size=2G --numjobs=1 --runtime=60 \
              --group_reporting --directory=/volume
          echo ""
          echo "=== Sequential Write Bandwidth (128K) ==="
          fio --name=seq-write --ioengine=libaio --iodepth=64 --rw=write \
              --bs=128k --direct=1 --size=2G --numjobs=1 --runtime=60 \
              --group_reporting --directory=/volume
          echo ""
          echo "=== Mixed Random Read/Write IOPS (4K, 75/25) ==="
          fio --name=mixed-rw --ioengine=libaio --iodepth=64 --rw=randrw \
              --rwmixread=75 --bs=4k --direct=1 --size=2G --numjobs=1 \
              --runtime=60 --group_reporting --directory=/volume
          echo ""
          echo "=== Benchmark Complete ==="
        volumeMounts:
        - name: vol
          mountPath: /volume
      restartPolicy: Never
      volumes:
      - name: vol
        persistentVolumeClaim:
          claimName: kbench-pvc-longhorn
  backoffLimit: 0
EOF

# Deploy the benchmark
kubectl apply -f kbench-longhorn.yaml

# Watch progress (benchmark takes approximately 5-7 minutes)
kubectl logs -f job/kbench-longhorn
```

When the job completes, the logs will show detailed FIO output for each test profile including IOPS, bandwidth (BW), and latency percentiles. Record these results before cleaning up.

```
# Clean up after the benchmark
kubectl delete -f kbench-longhorn.yaml
# Wait for PVC to fully delete before running another test
kubectl get pvc | grep kbench
```

2.3 Run a Comparison Benchmark (Longhorn vs Local-Path)

This is the recommended benchmark as it directly shows the overhead of Longhorn's distributed replication compared to raw local NVMe performance. Run the single benchmark twice — once with each storage class — and compare results.

```
# Step 1: Benchmark local-path (baseline NVMe performance)
cat <<'EOF' > kbench-localpath.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: kbench-pvc-localpath
spec:
  storageClassName: local-path
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 30Gi
---
apiVersion: batch/v1
kind: Job
metadata:
  name: kbench
spec:
  template:
    metadata:
      labels:
        kbench: fio
    spec:
      containers:
      - name: kbench
        image: alpine:latest
        command: ["/bin/sh", "-c"]
        args:
        - |
          apk add --no-cache fio
          echo "=== LOCAL-PATH Storage Benchmark ==="
          echo "=== Random Read IOPS (4K) ==="
          fio --name=rand-read --ioengine=libaio --iodepth=64 --rw=randread \
              --bs=4k --direct=1 --size=2G --numjobs=1 --runtime=60 \
              --group_reporting --directory=/volume
          echo ""
          echo "=== Random Write IOPS (4K) ==="
          fio --name=rand-write --ioengine=libaio --iodepth=64 --rw=randwrite \
              --bs=4k --direct=1 --size=2G --numjobs=1 --runtime=60 \
              --group_reporting --directory=/volume
          echo ""
          echo "=== Sequential Read Bandwidth (128K) ==="
          fio --name=seq-read --ioengine=libaio --iodepth=64 --rw=read \
              --bs=128k --direct=1 --size=2G --numjobs=1 --runtime=60 \
              --group_reporting --directory=/volume
          echo ""
          echo "=== Sequential Write Bandwidth (128K) ==="
          fio --name=seq-write --ioengine=libaio --iodepth=64 --rw=write \
              --bs=128k --direct=1 --size=2G --numjobs=1 --runtime=60 \
              --group_reporting --directory=/volume
          echo ""
```

```
          echo "=== Mixed Random Read/Write IOPS (4K, 75/25) ==="
          fio --name=mixed-rw --ioengine=libaio --iodepth=64 --rw=randrw \
              --rwmixread=75 --bs=4k --direct=1 --size=2G --numjobs=1 \
              --runtime=60 --group_reporting --directory=/volume
          echo ""
          echo "=== Benchmark Complete ==="
        volumeMounts:
        - name: vol
          mountPath: /volume
      restartPolicy: Never
      volumes:
      - name: vol
        persistentVolumeClaim:
          claimName: kbench-pvc-localpath
  backoffLimit: 0
EOF

# Run local-path benchmark
kubectl apply -f kbench-localpath.yaml
kubectl logs -f job/kbench-localpath
# Save the output, then clean up
kubectl delete -f kbench-localpath.yaml

# Step 2: Run the Longhorn benchmark (Section 2.2 manifest)
kubectl apply -f kbench-longhorn.yaml
kubectl logs -f job/kbench
kubectl delete -f kbench-longhorn.yaml

# Compare the IOPS and bandwidth numbers side by side
```

By comparing the two sets of results, you can directly see the overhead Longhorn's replication adds. Expect Longhorn write IOPS and bandwidth to be lower than local-path due to network replication to multiple nodes.

2.4 Understanding the Results

The kbench output reports three key metrics for each test profile:

IOPS (I/O Operations Per Second): Measures how many small (4K) read or write operations the storage can handle per second. Higher is better. This metric is critical for database workloads, etcd, and any application that performs many small random reads/writes.

Bandwidth (Throughput): Measures how much data (in MB/s) the storage can transfer using larger (128K) block sizes. Higher is better. This matters for sequential workloads like log streaming, backups, and large file transfers.

Latency: The time each I/O request takes to complete, measured in microseconds. Lower is better. High latency directly impacts application response times.

For the comparison benchmark, you will see side-by-side results for local-path and longhorn. A typical distributed storage overhead for Longhorn with 3 replicas on 1Gbps networking is 2–4× lower IOPS and bandwidth compared to local storage. On the RK1's

1Gbps backplane, the network is usually the bottleneck for write-heavy workloads since each write must be replicated to multiple nodes.

Tip — PVC size matters: For accurate results, the test file size should be at least 25× the read/write bandwidth to avoid filesystem and kernel caching effects. The default 30Gi PVC is appropriate for NVMe drives.

3. Control Plane Benchmark

This section measures Kubernetes control plane responsiveness by timing pod lifecycle operations using standard kubectl commands. No additional tools or dependencies are required — just kubectl access to the cluster.

3.1 What This Measures

The control plane benchmark answers key questions about cluster responsiveness: How fast can the cluster schedule and start pods? How quickly does a deployment scale up and down? How long does cleanup take? These measurements reflect the combined performance of the API server, etcd, the scheduler, and the kubelet across all nodes.

These measurements are especially interesting on the RK1's ARM64 architecture, as they show how the RK3588's big.LITTLE CPU design (Cortex-A76 performance cores + Cortex-A55 efficiency cores) handles Kubernetes scheduling and API operations.

3.2 Run the Control Plane Benchmark

This script creates an NGINX deployment, scales it up and down, and measures the time for each operation. Run from any node with kubectl access:

```
#!/bin/bash
# control-plane-bench.sh - Kubernetes Control Plane Benchmark

echo "=== Control Plane Benchmark ==="

# --- Test 1: Cold Start (includes image pull) ---
echo ""
echo "=== Test 1: Deploy 25 pods (cold start, includes image pull) ==="
START=$(date +%s%N)
kubectl create deployment bench-nginx --image=nginx --replicas=25
kubectl rollout status deployment/bench-nginx --timeout=300s
END=$(date +%s%N)
ELAPSED=$(( (END - START) / 1000000 ))
echo "Time to deploy 25 pods: ${ELAPSED}ms"

# Show pod distribution across nodes
echo ""
echo "=== Pod Distribution ==="
```

```
kubectl get pods -l app=bench-nginx -o wide --no-headers | \
  awk '{print $7}' | sort | uniq -c

# --- Test 2: Scale Out (images cached) ---
echo ""
echo "=== Test 2: Scale 25 -> 50 pods (images cached) ==="
START=$(date +%s%N)
kubectl scale deployment/bench-nginx --replicas=50
kubectl rollout status deployment/bench-nginx --timeout=300s
END=$(date +%s%N)
ELAPSED=$(( (END - START) / 1000000 ))
echo "Time to scale to 50 pods: ${ELAPSED}ms"

# --- Test 3: Scale In ---
echo ""
echo "=== Test 3: Scale 50 -> 5 pods ==="
START=$(date +%s%N)
kubectl scale deployment/bench-nginx --replicas=5
kubectl wait --for=condition=available deployment/bench-nginx --timeout=120s
END=$(date +%s%N)
ELAPSED=$(( (END - START) / 1000000 ))
echo "Time to scale to 5 pods: ${ELAPSED}ms"

# --- Test 4: Cleanup ---
echo ""
echo "=== Test 4: Delete deployment ==="
START=$(date +%s%N)
kubectl delete deployment bench-nginx --wait=true
END=$(date +%s%N)
ELAPSED=$(( (END - START) / 1000000 ))
echo "Time to delete deployment: ${ELAPSED}ms"

echo ""
echo "=== Benchmark Complete ==="
```

You can save this as a script and run it, or paste the commands directly. The benchmark takes approximately 1–2 minutes to complete.

3.3 Understanding the Results

Cold start deployment (Test 1): This is the slowest test because it includes pulling the nginx container image to each node for the first time. On subsequent runs (or after the first test), images are cached and pod startup is much faster. Expect 25–40 seconds for 25 pods on a cold start across 4 nodes.

Scale-out (Test 2): With images already cached, scaling from 25 to 50 pods measures pure scheduling and container creation speed. Expect 2–5 seconds since no image pulls are needed.

Scale-in (Test 3): Measures how quickly the cluster can terminate pods. This is typically sub-second as Kubernetes sends SIGTERM and moves on.

Cleanup (Test 4): Measures how quickly the cluster deletes the deployment and its remaining pods. Also typically sub-second.

Pod distribution: Shows how the scheduler balances pods across nodes. With the default scheduler, pods should be roughly evenly distributed across all 4 nodes.

Tip — repeat for warm results: Run the benchmark twice. The first run includes image pull time; the second run with cached images shows the true control plane performance. You can also increase the pod count to 100 to stress-test the cluster more aggressively.

4. Security Benchmark: Aqua kube-bench

kube-bench (github.com/aquasecurity/kube-bench) is the standard tool for auditing Kubernetes cluster security against the CIS Kubernetes Benchmark. It checks configuration files, process arguments, permissions, and policies against security best practices. kube-bench includes specific benchmark profiles for K3s.

4.1 What kube-bench Checks

The CIS Kubernetes Benchmark covers five major areas: control plane component configuration (API server, scheduler, controller manager), etcd configuration, control plane general configuration (authentication, authorization, logging), worker node configuration (kubelet, container runtime), and Kubernetes policies (RBAC, pod security, network policies, secrets management). Each check results in PASS, FAIL, or WARN, with specific remediation steps for failures.

4.2 Run kube-bench on Server Nodes

K3s bundles its components differently than standard Kubernetes, so kube-bench needs the --benchmark flag to use the K3s-specific CIS profile. Run directly on a server node:

```
# Run kube-bench using the K3s CIS benchmark profile
# On any server node (k3-node1, k3-node2, or k3-node3):
curl -L
https://github.com/aquasecurity/kube-bench/releases/download/v0.14.0/kube-bench_0.14
.0_linux_arm64.tar.gz -o kube-bench.tar.gz
tar -xzf kube-bench.tar.gz
sudo mkdir -p /opt/kube-bench
sudo mv kube-bench /usr/local/bin/
sudo cp -r cfg /opt/kube-bench/cfg

# Run all CIS benchmark sections (master, etcd, controlplane, node, policies)
sudo kube-bench run --benchmark k3s-cis-1.7 --config-dir /opt/kube-bench/cfg \
  --targets master,etcd,controlplane,node,policies

# Save results to a file
sudo kube-bench run --benchmark k3s-cis-1.7 --config-dir /opt/kube-bench/cfg \
  --targets master,etcd,controlplane,node,policies > kube-bench-server-results.txt
2>&1
cat kube-bench-server-results.txt
```

4.3 Run kube-bench on the Worker Node

The worker node has a different profile since it does not run control plane components:

```
# On k3-node4 (worker):
# Install kube-bench using the same steps as Section 4.2, then:
sudo kube-bench run --benchmark k3s-cis-1.7 --config-dir /opt/kube-bench/cfg \
  --targets node

# Save results
sudo kube-bench run --benchmark k3s-cis-1.7 --config-dir /opt/kube-bench/cfg \
  --targets node > kube-bench-worker-results.txt 2>&1
cat kube-bench-worker-results.txt
```

4.4 Run kube-bench as a Kubernetes Job

Alternatively, you can run kube-bench as a Kubernetes Job. However, the default job.yaml is designed for standard kubeadm clusters. For K3s, you need a modified job that adds the K3s-specific flags and volume paths:

```
# Create a K3s-aware kube-bench job
cat <<'EOF' > kube-bench-k3s-job.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: kube-bench
spec:
  template:
    metadata:
      labels:
        app: kube-bench
    spec:
      containers:
      - name: kube-bench
        image: docker.io/aquasec/kube-bench:v0.14.0
        command: ["kube-bench", "run", "--benchmark", "k3s-cis-1.7", "--config-dir",
"/opt/kube-bench/cfg", "--targets", "master,etcd,controlplane,node,policies"]
        volumeMounts:
        - name: var-lib-rancher
          mountPath: /var/lib/rancher
          readOnly: true
        - name: etc-rancher
          mountPath: /etc/rancher
          readOnly: true
        - name: etc-systemd
          mountPath: /etc/systemd
          readOnly: true
        - name: lib-systemd
          mountPath: /lib/systemd
          readOnly: true
        - name: var-lib-kubelet
          mountPath: /var/lib/kubelet
          readOnly: true
        - name: etc-cni-netd
          mountPath: /etc/cni/net.d/
```

```
          readOnly: true
      hostPID: true
      restartPolicy: Never
      tolerations:
      - key: node-role.kubernetes.io/control-plane
        operator: Exists
        effect: NoSchedule
      volumes:
      - name: var-lib-rancher
        hostPath:
          path: /var/lib/rancher
      - name: etc-rancher
        hostPath:
          path: /etc/rancher
      - name: etc-systemd
        hostPath:
          path: /etc/systemd
      - name: lib-systemd
        hostPath:
          path: /lib/systemd
      - name: var-lib-kubelet
        hostPath:
          path: /var/lib/kubelet
      - name: etc-cni-netd
        hostPath:
          path: /etc/cni/net.d/
  backoffLimit: 0
EOF

# Deploy and retrieve results
kubectl apply -f kube-bench-k3s-job.yaml
kubectl wait --for=condition=complete job/kube-bench --timeout=120s
kubectl logs job/kube-bench

# Clean up
kubectl delete -f kube-bench-k3s-job.yaml
```

4.5 Understanding the Results

kube-bench output is organized into sections matching the CIS benchmark structure. Each check shows one of four statuses:

[PASS]: The configuration meets the CIS recommendation. No action needed.

[FAIL]: The configuration does not meet the CIS recommendation. A remediation step is provided.

[WARN]: The check could not be automatically verified and requires manual review.

[INFO]: Informational finding only.

At the end of the output, a summary table shows the total PASS/FAIL/WARN/INFO counts for each section. A freshly installed K3s cluster typically passes most checks by default, as K3s ships with sane security defaults. Common findings that may show FAIL or WARN include: audit logging not being enabled, admission controllers not configured, and network policies not being defined for all namespaces.

Important — not all FAILs require action: Some CIS recommendations may not apply to your use case or may conflict with your cluster's requirements. The CIS benchmark is a set of best practices, not a strict compliance requirement. Review each FAIL in context before remediating. For a home lab or development cluster, WARN-level findings are often acceptable.

5. Running a Complete Benchmark Suite

To get a comprehensive picture of your cluster, run all three benchmarks in sequence. The recommended order avoids interference between tests:

5.1 Recommended Order

Step 1 — kube-bench (Security): Run first because it's non-destructive and does not create any workloads. It only reads configuration and process state.

Step 2 — Longhorn kbench (Storage): Run second because storage benchmarks should be done when the cluster is otherwise idle. Any background I/O from other benchmarks would skew results.

Step 3 — Control Plane Benchmark: Run last because it creates and destroys many pods, which generates I/O and network activity. Wait for Longhorn volumes from Step 2 to be fully cleaned up before running.

5.2 Quick-Run Script

This script runs all three benchmarks in sequence and saves results to timestamped files:

```
#!/bin/bash
# benchmark-all.sh - Run all Kubernetes benchmarks
TIMESTAMP=$(date +%Y%m%d-%H%M%S)
RESULTS_DIR="benchmark-results-${TIMESTAMP}"
mkdir -p "${RESULTS_DIR}"
echo "=== Benchmark results will be saved to ${RESULTS_DIR} ==="

# Step 1: CIS Security Benchmark
echo ""
echo "=== [1/3] Running CIS Security Benchmark (kube-bench) ==="
sudo kube-bench run --benchmark k3s-cis-1.7 --config-dir /opt/kube-bench/cfg \
  --targets master,etcd,controlplane,node,policies >
"${RESULTS_DIR}/kube-bench-cis.txt" 2>&1
echo "CIS benchmark complete. Results: ${RESULTS_DIR}/kube-bench-cis.txt"
tail -5 "${RESULTS_DIR}/kube-bench-cis.txt"

# Step 2: Storage Benchmark
echo ""
echo "=== [2/3] Running Storage Benchmark (kbench) ==="
kubectl apply -f kbench-compare.yaml
```

```
echo "Waiting for storage benchmark to complete (approx 12-15 minutes)..."
kubectl wait --for=condition=complete job/kbench --timeout=1200s
kubectl logs job/kbench > "${RESULTS_DIR}/kbench-storage.txt" 2>&1
kubectl delete -f kbench-compare.yaml
echo "Storage benchmark complete. Results: ${RESULTS_DIR}/kbench-storage.txt"

# Step 3: Control Plane Benchmark
echo ""
echo "=== [3/3] Running Control Plane Benchmark ==="
bash control-plane-bench.sh > "${RESULTS_DIR}/control-plane.txt" 2>&1
echo "Control plane benchmark complete. Results: ${RESULTS_DIR}/control-plane.txt"

echo ""
echo "=== All benchmarks complete ==="
echo "Results saved in: ${RESULTS_DIR}/"
ls -la "${RESULTS_DIR}/"
```

5.3 Monitoring During Benchmarks

Since the cluster has Prometheus and Grafana installed (see deployment guide Section 12), you can observe system behavior during benchmarks in real time:

Grafana dashboard: Open http://192.168.4.202 and watch the Node Exporter / Nodes dashboard during benchmark runs. Pay attention to CPU utilization across the big.LITTLE cores, NVMe I/O latency, and network throughput.

Longhorn UI: Monitor volume I/O during the storage benchmark to see how replicas distribute write load.

Screenshots of the Grafana dashboards during benchmark runs provide valuable context for interpreting the numerical results.

6. Cleanup

After benchmarking, remove the tools and test artifacts:

```
# Remove storage benchmark manifests
rm -f kbench-longhorn.yaml kbench-localpath.yaml

# Remove control plane benchmark script
rm -f control-plane-bench.sh

# Remove kube-bench binary and config
sudo rm -f /usr/local/bin/kube-bench
sudo rm -rf /opt/kube-bench
rm -f kube-bench.tar.gz

# Remove kube-bench K3s job manifest
rm -f kube-bench-k3s-job.yaml

# Verify no benchmark pods remain
kubectl get pods -A | grep -E "kbench|kube-bench|bench-nginx"
```

```
# Check Longhorn volumes are cleaned up
kubectl get pvc -A | grep kbench
```

If any kbench PVCs remain in Terminating state, check that the Longhorn CSI plugin pods are running (see deployment guide Section 13.4 recovery checklist).